Chasing the "Aha Moment": A Review and Implementation of Reinforcement Learning in the Post-Training of LLMs

Ryan Li **Serena You** **Yuanyuan Zhu** **Diwesh Chaurasia**
Department of Computer Science and Department of Bioengineering
Rice University
{rl182, xy68, yz281, dc162}@rice.edu

Abstract

Reinforcement learning (RL) has become a central ingredient in the post-training of large language models (LLMs). RL-trained models such as DeepSeek-R1 achieve strong reasoning performance, and the DeepSeek-R1 report describes an "Aha Moment" in which the model learns to allocate more thinking time by revisiting and revising its initial reasoning. This phenomenon has drawn wide attention in the research community. In this project, we review RL-based post-training methods for LLMs and implement Group Relative Policy Optimization (GRPO) on Llama 3.1–8B Instruct under a single-GPU budget. We further design handcrafted reward functions, including a length-based term that rewards longer reasoning when an answer is wrong, to test whether such shaping can elicit behavior similar to the reported Aha Moment. Our experiments show that the GRPO-trained model performs well on GSM8K but fails on AIME. This contrast suggests that, at the 8B scale, RL does not fundamentally expand a model's underlying capability; it mainly encourages more structured and consistent reasoning. We discuss these findings and outline future work on scaling model size and refining reward design.

1 Introduction

Large language models (LLMs) have advanced rapidly in recent years and now play a central role in natural language processing, reasoning, and decision-making tasks. Scaling laws show that model capability improves predictably with data and compute [1, 2], yet post-training has become increasingly important for improving reasoning quality beyond what pretraining and supervised learning alone provide. Among post-training techniques, reinforcement learning (RL) has become a major direction for alignment, instruction following, and step-by-step reasoning [3, 4].

Recent RL-trained models such as DeepSeek-R1 [5] achieve strong results on difficult reasoning benchmarks. A notable observation from that work is the so-called "Aha Moment" (Figure 1), in which the model learns, without explicit supervision, to spend more time thinking by revisiting and correcting its initial reasoning. This observation has attracted significant attention because it suggests that RL can shape the reasoning *process* rather than only the final answer.

However, the mechanism by which RL improves reasoning remains unclear. Several studies argue that RL-based methods, including RLHF, RLAIF, DPO, and GRPO, do not fundamentally expand a model's representational capacity. Instead, they encourage stability, coherence, and more deliberate reasoning patterns that are already latent in the pretrained model [5, 6, 7]. Understanding these dynamics matters for designing reward functions that genuinely improve reasoning rather than overfit to shallow heuristics.

COMP 514 Final Project Report, Rice University, Fall 2025.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>
To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$
$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$.
Rearrange to isolate the inner square root term:
$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$
$\ldots$
Wait, wait. Wait. That's an aha moment I can flag here.
Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$
We started with the equation:
$\sqrt{a - \sqrt{a + x}} = x$
First, let's square both sides:
$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$
Next, I could square both sides again, treating the equation: $\cdots$
$\ldots$

Figure 1: The "Aha Moment" reported for DeepSeek-R1-Zero: during RL training, an intermediate checkpoint pauses, re-evaluates its approach, and revises its reasoning. Figure reproduced from the DeepSeek-R1 technical report [5].

In this project, we investigate the role of RL in post-training LLM reasoning. Our contributions are threefold:

- We review and summarize existing post-training techniques for LLMs, including supervised fine-tuning (SFT), RLHF, GRPO, and related preference-based and process-based methods.
- We implement GRPO on Llama 3.1–8B Instruct using 4-bit quantization and LoRA on a single NVIDIA T4 GPU, and we design handcrafted reward functions intended to encourage deeper reflection and potentially trigger Aha-Moment-like behavior.
- We evaluate the trained model on GSM8K and AIME. RL improves GSM8K accuracy by about 2.4 percentage points (82.6% to 85.0%) but does not transfer to AIME, supporting the hypothesis that RL encourages longer and more consistent chains of thought without increasing the inherent reasoning ability of the model.

These findings motivate our future work. We plan to scale up the base model and to explore whether more structured reward functions can meaningfully improve high-level mathematical reasoning, which may shed light on the limits of RL-based post-training and its interaction with model capacity.

Our code, training notebooks, and this report are available at `https://github.com/Ryanrc03/Chasing-the-Aha-Moment`.

2 Related Work

2.1 Supervised Fine-Tuning

Supervised fine-tuning (SFT) is the foundational post-training step used to align pretrained language models with human-preferred behavior. Popularized by InstructGPT [3], SFT trains on curated prompt–response pairs with a next-token prediction objective, so the model learns to imitate high-quality demonstrations in both content and structure. Instruction-tuned systems such as Llama 3 Instruct [8] show that SFT reliably turns a pretrained LLM, whose outputs often resemble uncontrolled text continuation, into a capable instruction follower with improved coherence, formatting, and task adherence. Because SFT requires neither a reward model nor preference comparisons, it remains the most stable and widely used first stage in modern LLM pipelines.

Stabilizing the model before RL. A key role of SFT in the DeepSeek-R1 pipeline is to stabilize the model before reinforcement learning. The authors report that DeepSeek-R1-Zero, trained purely with RL from the base model, frequently produced unreadable chains of thought, fragmented or looping sentences, and mixed English–Chinese token streams [5]. These issues indicate that RL applied directly to a pretrained model can amplify noise and stylistic inconsistencies rather than produce meaningful improvement. A small amount of "cold-start" SFT lets DeepSeek-R1 avoid such degenerate behavior: the model learns clean formatting, coherent multi-sentence reasoning, and a consistent separation between the reasoning process and the final summary, giving RL a well-structured linguistic foundation.

Providing a reasoning scaffold. Although SFT does not itself induce the Aha Moment, it establishes the structural scaffold on which such behavior can emerge. Through demonstrations of step-by-step explanations and final summaries, SFT teaches the model the *format* of a reasoning process, even if the model does not yet refine or critique its own steps. DeepSeek-R1 formalizes this scaffold with a template that has explicit reasoning and summary regions. RL then uses the scaffold to encourage self-correction, re-evaluation, and adaptive thinking depth. In this sense, SFT provides the grammatical and organizational framework within which RL can later induce reflective reasoning.

Limitations. SFT is fundamentally imitation-based and therefore has intrinsic limits. Because it optimizes only the likelihood of demonstrated outputs, it does not reward deeper reasoning, exploration, or alternative solution strategies. It cannot teach a model to reflect on or correct its own mistakes, nor to modulate thinking time according to task difficulty. SFT-trained models also tend to overfit the stylistic patterns of the demonstration data, which limits their ability to discover new reasoning approaches. For these reasons, SFT alone is insufficient for producing emergent behaviors such as the Aha Moment, motivating preference-based and reinforcement-based methods that optimize reasoning quality rather than pure imitation.

2.2 Reinforcement Fine-Tuning

The 2017 work of Christiano et al. [9], which first used human preference signals to train a reward model that then guides policy optimization, is widely regarded as the conceptual origin of reinforcement fine-tuning (RFT). Instead of hand-crafted reward signals, it collects pairwise comparisons of short trajectory segments from human evaluators, trains a reward model on these comparisons, and uses that model to guide an RL agent. Building on this framework, Ziegler et al. [4] showed that the same methodology extends to large language models, giving rise to the RLHF pipeline widely used today: collect human preference data, fit a reward model, and optimize the policy with Proximal Policy Optimization (PPO) under a KL penalty that keeps updates stable. This work established RLHF as a practical and effective method for aligning language models with human expectations [3, 10].

Modern RFT methods extend RLHF with more stable, reward-model-free, and compute-efficient optimization schemes that better support large-scale reasoning. Among them, Group Relative Policy Optimization (GRPO) [11] is a central development. GRPO samples a group of candidate solutions for each problem, scores them with task-specific correctness signals, and uses the group statistics as a baseline, which removes the need for a separate value network and reduces the training instability commonly seen in PPO-style RLHF. In DeepSeekMath and DeepSeek-R1, GRPO is applied as a reinforcement fine-tuning stage after supervised learning to strengthen multi-step mathematical reasoning [11, 5]. Figure 2 gives an overview of the general RL-for-LLM framework.

More generally, RFT improves a model by training on a reward signal rather than only on labeled data. The model produces multiple candidate responses, which are assessed by human feedback, AI feedback, deterministic correctness checks, or group-wise ranking. These assessments yield scalar or relative rewards that are converted into advantages for policy-gradient updates such as PPO or GRPO. Through repeated updates, the model increases the likelihood of high-quality reasoning trajectories and better-aligned outputs.

Advantages and disadvantages. RFT can incorporate evaluative feedback, whether from humans, AI feedback models, or deterministic verifiers, into a principled optimization framework. This lets it optimize behaviors that supervised learning cannot easily capture, such as multi-step reasoning, self-

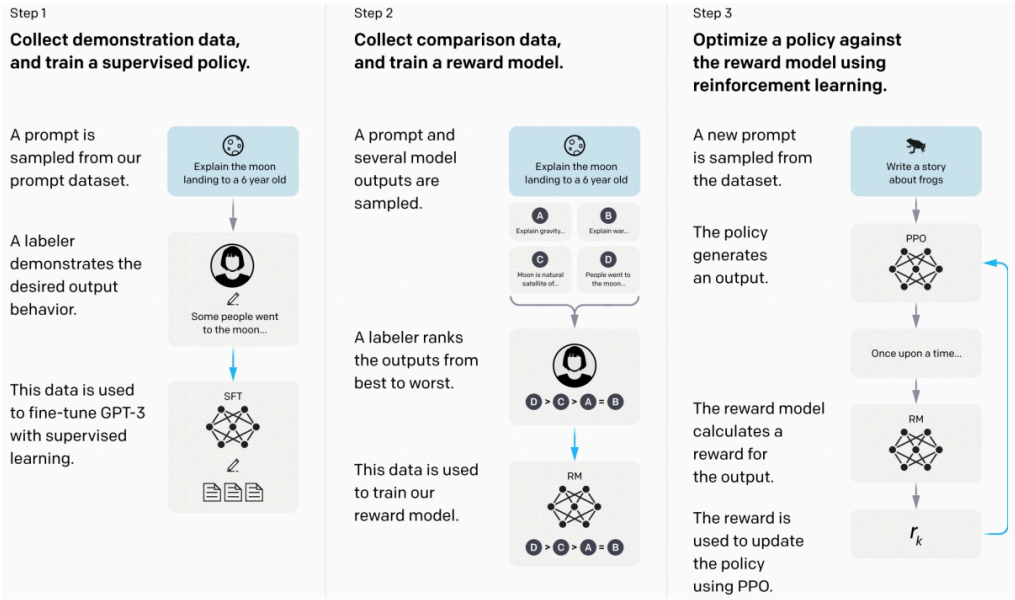


Figure 2: Overview of the RL-for-LLMs framework. Adapted from *Reinforcement Learning Enhanced LLMs: A Survey* [12].

correction, tool use, and structured task completion. On the other hand, RLHF-style RFT requires large amounts of human preference data, making it expensive and sensitive to annotation quality [10]. Learned reward models are prone to over-optimization, where the policy exploits weaknesses of the reward model rather than genuinely improving, leading to instability and reward hacking [13]. Even reward-model-free variants incur substantial compute, because they need multiple samples per prompt and additional evaluation overhead. Finally, the policy-gradient nature of RFT brings optimization challenges, including sensitivity to hyperparameters and reduced output diversity when preference shaping becomes too strong [13].

Cost. The cost of RFT arises from data collection, sampling, evaluation, and policy optimization. Among SFT, RLHF, and GRPO, RLHF is the most expensive: it relies on large-scale human preference data and a separately trained reward model, and the policy must repeatedly query that reward model during PPO updates, creating a two-model training loop [10, 13]. GRPO reduces cost substantially by eliminating the reward model. It replaces a learned reward function with deterministic correctness signals or group-wise rankings over sampled outputs, removing the need for human annotators and reward-model training, which is what makes it feasible on a single consumer-grade GPU as in this project.

2.3 Reproducing the Aha Moment at Small Scale

Several recent works ask whether Aha-Moment-like behavior can be reproduced in much smaller models through RL post-training. Zhou et al. [6] report self-reflective reasoning emerging in a 2B non-SFT multimodal model trained with GRPO. TinyLLaVA-Video-R1 [14] applies RL to a 4B video model and observes reflective reasoning and "rethinking" during training. SimpleRL-Zoo [15] studies zero-RL training across ten open base models from 0.5B to 32B parameters and finds that increases in response length and reflective phrases appear in several of them, while also cautioning that such surface signals do not always coincide with accuracy gains. Yang et al. [7] analyze Aha Moments from the perspective of internal mechanisms and argue that they reflect changes in how the model allocates computation rather than new knowledge. These works motivate our attempt to elicit similar behavior in an 8B model under a tight compute budget.

3 Methodology

In this section we describe the output format and the notion of "reasoning" used throughout our experiments, the GRPO objective that connects rewards to model updates, and the reward design we use to encourage reflective behavior.

3.1 Reasoning as Rewarded Trajectories

Following chain-of-thought (CoT) prompting [16, 17], we structure the model's output as an explicit reasoning trace followed by a final answer. For a question x the model generates $y = (z, a)$, where z is the chain-of-thought segment and a is the final answer. In practice we realize this structure with XML-style tags:

$$y \;=\; \texttt{<reasoning>}\; z \;\texttt{</reasoning><answer>}\; a \;\texttt{</answer>}. \tag{1}$$

At evaluation time we parse a from the `<answer>` tag and compare it with the gold answer, while z is used to analyze reasoning behavior such as the length and stability of the chain of thought.

We treat the language model as a policy $\pi_\theta(\cdot \mid x)$ that samples trajectories $\tau = (z, a)$, and we score each trajectory with a scalar reward

$$R(x, \tau) \;=\; R_{\text{acc}}(x, a) \;+\; R_{\text{format}}(x, \tau) \;+\; R_{\text{len}}(x, z), \tag{2}$$

where R_{acc} rewards a correct final answer, R_{format} rewards well-formed XML structure with a clean separation between reasoning and answer, and R_{len} is a length-based term, defined in Section 3.3, that encourages longer and more deliberate reasoning when the model would otherwise terminate prematurely or answer incorrectly. "Reasoning" is thus operationalized as the set of trajectories that obtain high reward under R, rather than as a separate symbolic module.

In our implementation, R_{acc} gives 2.0 for an exact match with the gold answer and an additional 0.5 when the extracted answer is an integer. R_{format} is the sum of a strict and a soft regular-expression match on the tag structure (0.5 each) and a fine-grained tag-count term that awards 0.125 per correctly placed tag and penalizes trailing text after `</answer>`.

3.2 GRPO-Based Fine-Tuning

To connect the reward to model updates we adopt Group Relative Policy Optimization (GRPO) [11], a variant of PPO that removes the need for a separate value network. For each question x we sample a group of K candidate trajectories $\{\tau_k\}_{k=1}^{K}$ from the old policy $\pi_{\theta_{\text{old}}}(\cdot \mid x)$ and compute their rewards $r_k = R(x, \tau_k)$. GRPO uses the group mean as a baseline,

$$\bar{r} \;=\; \frac{1}{K}\sum_{k=1}^{K} r_k, \qquad A_k \;=\; r_k - \bar{r}, \tag{3}$$

and optimizes a clipped sequence-level objective

$$J_{\text{GRPO}}(\theta) \;=\; \mathbb{E}_x\left[\frac{1}{K}\sum_{k=1}^{K} \min\Big(\rho_k(\theta)A_k, \text{clip}\big(\rho_k(\theta), 1-\varepsilon, 1+\varepsilon\big)A_k\Big)\right], \tag{4}$$

where

$$\rho_k(\theta) \;=\; \frac{\pi_\theta(\tau_k \mid x)}{\pi_{\theta_{\text{old}}}(\tau_k \mid x)} \tag{5}$$

is the importance ratio between the current and old policies and ε is the clipping hyperparameter. Intuitively, GRPO raises the probability of trajectories whose reward exceeds the group average and lowers that of trajectories below it, while clipping stabilizes the update. The original formulation also normalizes A_k by the group standard deviation and adds a KL penalty toward a reference policy; we follow the TRL implementation, which includes both.

Setting	Value
Base model	Llama 3.1–8B Instruct, 4-bit
LoRA rank / α	32 / 32
LoRA targets	q, k, v, o, gate, up, down projections
Generations per prompt (K)	6
Max prompt / completion length	256 / 768 tokens
Optimizer	paged AdamW (8-bit), β_1=0.9, β_2=0.99, wd 0.1
Learning rate	5×10^{-6}, cosine, 10% warm-up
Gradient clipping	0.1
Training steps	250

Table 1: GRPO training configuration used in all experiments.

3.3 Length-Based Reward Shaping

One of the most salient empirical signatures of the Aha Moment is the emergence of longer and more deliberate chains of thought, especially when the model's initial attempt is incorrect. To encourage such reflective behavior explicitly, we augment the reward with a length-based component that depends on the chain-of-thought segment z.

Let $\ell(z)$ be the length of z in tokens and $L_{\max}$ a fixed truncation threshold. We define the normalized length

$$\tilde{\ell}(z) \;=\; \min\Big(\frac{\ell(z)}{L_{\max}},\, 1\Big) \in [0,1], \tag{6}$$

which measures how close the reasoning trace is to the target length scale. Writing $\mathbb{I}(\cdot)$ for the indicator function and treating R_{acc} as a binary correctness signal, the length-based term is

$$R_{\text{len}}(x,z) \;=\; \alpha\,\mathbb{I}\big(R_{\text{acc}}(x,a)=0\big)\,\tilde{\ell}(z) \;-\; \beta\,\mathbb{I}\big(R_{\text{acc}}(x,a)=1\big)\,\tilde{\ell}(z), \tag{7}$$

with hyperparameters $\alpha > 0$ and $\beta \geq 0$.

When the final answer is wrong, the model receives a bonus proportional to the normalized CoT length, which encourages it to allocate more "thinking time" by extending its reasoning. When the answer is already correct, overly long chains of thought are mildly penalized, which discourages unbounded verbosity. The overall reward in Equation 2 therefore encodes a preference for reflective behavior: in failure states the model is pushed toward deeper reasoning, and in success states it is nudged toward concise but well-structured solutions. In practice we choose modest values of α and β so that R_{len} shapes the trajectory without overwhelming the accuracy signal.

4 Experiments

4.1 Experimental Setup

All experiments run in Google Colab on a single NVIDIA T4 GPU with 15 GB of memory. Our policy is Llama 3.1–8B Instruct [8] loaded through Unsloth [18] in 4-bit quantization [19], with a rank-32 LoRA adapter [20] on all attention and feed-forward projections (about 84M trainable parameters, or 1% of the model). Generation during training uses vLLM 0.9.2 co-located with the trainer, and policy updates use the GRPO implementation in TRL 0.22.2 [21] with PyTorch 2.7 and CUDA 12.6.

Table 1 lists the main hyperparameters. We sample six completions per prompt, cap prompts at 256 tokens and completions at 768 tokens, and train for 250 steps with 8-bit paged AdamW, a learning rate of 5×10^{-6} with cosine decay and 10% warm-up, and gradient clipping at 0.1. Random seeds are fixed. Despite the limited hardware, this configuration reliably supports controlled studies of GRPO and reward shaping.

4.2 Datasets

All experiments use public mathematical reasoning datasets. For RL training we use the `openai/gsm8k` training split [22], converting each example into a chat-style prompt with the sys-

tem instruction that specifies the XML output format, and extracting the gold answer after the #### marker. GSM8K's concise, well-structured grade-school word problems make it an effective setting for studying small-model RL.

For evaluation we use two benchmarks: (1) the full GSM8K test split (1,319 problems) to measure in-distribution performance, and (2) a combined AIME set of 60 problems, consisting of AIME 2024 and AIME 2025 (parts I and II), to assess out-of-distribution generalization.[1] AIME problems are substantially harder and require longer and more precise reasoning chains. Evaluation uses greedy-leaning sampling (temperature 0.2, top-p 0.95) with up to 512 generated tokens, and we report accuracy after numeric normalization of the extracted answer, the rate at which an `<answer>` tag could be parsed, and the rate at which the full `<reasoning>`/`<answer>` structure was matched.

4.3 Instruct Baseline vs. GRPO

The instruction-tuned starting point already provides a useful initialization for mathematical reasoning, but its behavior is limited by exposure bias and the absence of explicit reasoning supervision. In our experiments the baseline model often produces verbose or meandering explanations, switches solution strategies within a single response, or introduces unnecessary computational methods. Although it generates mathematically relevant text, it frequently fails to deliver a stable, well-formed final answer in the required `<answer>` format, treating the task as open-ended text generation rather than goal-directed problem solving.

GRPO changes this behavior substantially. Unlike SFT, which optimizes next-token prediction, GRPO directly reinforces correct final answers and a structured output format and penalizes inconsistent intermediate reasoning. Even a small amount of GRPO training yields noticeable improvements: the model becomes more concise, takes fewer detours in its reasoning, and reliably terminates with a correctly formatted answer. GRPO also reduces generation variability, making the model less prone to contradictory steps or mid-solution strategy changes. These gains appear despite the strict memory constraints of a single T4 GPU, which shows that reward-driven alignment can be effective at small scale.

Qualitatively, GRPO produces behavior reminiscent of the "micro-aha" phenomenon discussed in recent work on RL-enhanced LLMs. Whereas the baseline occasionally attempts several reasoning paths within one response, the GRPO-trained model follows a more deliberate pattern: it selects a single method, applies it consistently, and stays aligned with the task objective. This contrast is especially clear in our pi-calculation case study (Table 2), where the baseline gives long pedagogical explanations or switches algorithms mid-solution, while the GRPO-trained model follows one coherent line of reasoning and outputs a precise, well-structured final answer. Together, these observations highlight the role of reinforcement feedback in stabilizing reasoning behavior and improving task adherence beyond what SFT alone provides.

4.4 Rewarding Reflective CoT Expansion

Prior studies of the Aha Moment observe that models shift into longer and more deliberate chains of thought when their initial reasoning appears incorrect, a behavior interpreted as a self-reflective adjustment that allocates extra thinking time. Motivated by this, we test whether the length-based reward of Section 3.3 can encourage a similar reflective response in an 8B model.

Tables 3 and 4 compare GRPO with the base reward ($R_{\text{acc}} + R_{\text{format}}$) against GRPO with the full shaped reward of Equation 2. On GSM8K, reward shaping raises accuracy from 82.6% to 85.0% and increases the fraction of outputs matching the complete `<reasoning>`/`<answer>` structure from 0% to 23%, at the cost of a small increase in generation latency, consistent with longer reasoning traces. The near-perfect XML parse rate in both settings shows that the baseline reward already teaches the model to emit an `<answer>` tag; the shaped reward is what makes the surrounding reasoning structure consistent.

[1] `HuggingFaceH4/aime_2024` and `opencompass/AIME2025` on the Hugging Face Hub.

Instruct baseline (no GRPO)
`"Calculating pi is a classic mathematical challenge. There are various methods to calculate pi, but I'll use the most common and simple one, the Leibniz formula..."` The model outputs two different Python implementations (Leibniz series and Gauss–Legendre), discusses convergence, and adds extended mathematical commentary. No final numerical answer is given in the required `<answer>` format.
GRPO-trained model

`"Unfortunately, calculating the exact value of pi is a complex task... However, we can approximate it using the Leibniz series..."`
The model gives a concise explanation, a single coherent Python snippet, and a final answer in the required structured format:
`<answer>3.141592653589793</answer>`

Table 2: Qualitative comparison before and after GRPO training on the prompt "Calculate pi." Without GRPO the model rambles, switches methods mid-solution, and fails to produce a structured answer. After GRPO it selects one method, follows a stable reasoning pattern, and reliably closes with a clean `<answer>` tag.

Metric	**GRPO (base reward)**	**GRPO + reward shaping**
GSM8K test samples	1,319	1,319
Accuracy	0.8256	**0.8500**
XML parse rate	1.0000	0.9980
Format (soft)	0.0000	**0.2300**
Avg. latency (s / sample)	5.56	5.84
Wall-clock time (s)	7,329	7,700

Table 3: Effect of reward shaping on the GSM8K test set. The shaped reward yields a modest but consistent accuracy gain and a large gain in format compliance, with slightly higher generation latency.

4.5 Why GRPO Does Not Transfer to AIME

Despite clear improvements on GSM8K, our GRPO-trained 8B model does not generalize to AIME 2024/2025, where accuracy stays at zero. Although both benchmarks involve mathematical reasoning, AIME problems require multi-step symbolic manipulation, non-linear algebra, and olympiad-level abstraction far outside the GSM8K distribution. The absence of correct answers does not mean the model has not changed: we observe more deliberate chains of thought and consistently well-formed XML output. The model attempts deeper reasoning but lacks the capacity to complete these problems.

This matches common critiques of GRPO and related RL approaches: they refine and stabilize the model's existing reasoning behavior but do not expand its underlying algorithmic or representational ability. In effect, GRPO enforces more consistent reasoning trajectories without creating new mathematical competence, an inherent limitation at the 8B scale. The poor AIME performance therefore reflects the constraints of model scale and training budget rather than a flaw in the reward design. Larger models trained with GRPO or R1-style methods show more transferable reasoning, likely because of richer inductive biases and stronger latent mathematical priors. Within the small-model regime, our results mark a realistic boundary: GRPO can encourage more thoughtful reasoning, but it cannot compensate for insufficient capacity on high-difficulty tasks.

4.6 Discussion and Future Work

The sharp contrast between gains on GSM8K and zero accuracy on AIME highlights a core limitation of small LLMs trained with RL: RL improves consistency but does not expand the model's

Metric	GRPO (base reward)	GRPO + reward shaping
AIME samples	60	60
Accuracy	0.0000	0.0000
XML parse rate	1.0000	0.9833
Format (soft)	0.0333	**0.1167**
Avg. latency (s / sample)	14.26	15.48
Wall-clock time (s)	422.3	456.0

Table 4: GRPO with and without reward shaping on the combined AIME 2024 and 2025 set. Neither model solves any problem, reflecting the extreme difficulty and out-of-distribution nature of olympiad problems for an 8B model. Reward shaping nevertheless increases format consistency and generation time, indicating more structured and deliberate chains of thought even when the problems are not solved.

underlying reasoning capacity. GSM8K lies close to the pretraining distribution, whereas AIME demands multi-step symbolic manipulation and olympiad-level abstraction. Prior work [6, 5, 15] similarly finds that GRPO-like methods stabilize existing reasoning patterns without enabling fundamentally new capabilities.

This gap indicates that reward shaping alone cannot bridge the distributional jump from grade-school to olympiad mathematics. Model scale and pretraining depth appear decisive, as larger R1-style or process-supervised models [5, 12] exhibit more transferable reasoning. Scaling our experiments to 13B–70B checkpoints would clarify whether the benefits we observe persist beyond GSM8K.

A complementary direction is *process-level reward* [23], which offers denser supervision and may help smaller models build more verifiable reasoning chains. Synthetic data, self-play, and bootstrapping [5, 6, 17] could further enrich the training distribution and narrow the gap. Our own training budget (250 GRPO steps, roughly one GPU-hour) is also small; longer runs with more generations per prompt and longer completion limits are a natural next step, and we are preparing a follow-up run that compares a 3B base model trained for longer against the present 8B setting.

In summary, while GRPO improves structure and stability, our results reinforce that scale remains the dominant driver of out-of-distribution mathematical reasoning. Combining larger models, process-based rewards, and richer reasoning curricula may yield more generalizable improvements.

References

[1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[3] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

[4] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[6] Hengguang Zhou, Xirui Li, Ruochen Wang, Minhao Cheng, Tianyi Zhou, and Cho-Jui Hsieh. R1-Zero's "Aha Moment" in visual reasoning on a 2B Non-SFT model. *arXiv preprint arXiv:2503.05132*, 2025.

[7] Shu Yang, Junchao Wu, Xin Chen, Yunze Xiao, Xinyi Yang, Derek F. Wong, and Di Wang. Understanding Aha Moments: From external observations to internal mechanisms. *arXiv preprint arXiv:2504.02956*, 2025.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[9] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 30, 2017.

[10] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

[11] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yang Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[12] Shuhe Wang, Shengyu Zhang, Jie Zhang, Runyi Hu, Xiaoya Li, Tianwei Zhang, Jiwei Li, Fei Wu, Guoyin Wang, and Eduard Hovy. Reinforcement learning enhanced LLMs: A survey. *arXiv preprint arXiv:2412.10400*, 2024.

[13] Rafael Rafailov, Yaswanth Chittepu, Ryan Park, Harshit Sushil Sikchi, Joey Hejna, Bradley Knox, Chelsea Finn, and Scott Niekum. Scaling laws for reward model overoptimization in direct alignment algorithms. *Advances in Neural Information Processing Systems*, 37, 2024.

[14] Xingjian Zhang, Siwei Wen, Wenjun Wu, and Lei Huang. TinyLLaVA-Video-R1: Towards smaller LMMs for video reasoning. *arXiv preprint arXiv:2504.09641*, 2025.

[15] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

[16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

[17] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35, 2022.

[18] Daniel Han, Michael Han, and Unsloth team. Unsloth: Fast and memory-efficient LLM finetuning. `https://github.com/unslothai/unsloth`, 2023.

[19] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.

[20] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

[21] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[22] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[23] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[24] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023.

A Unused Related Work Draft (Contributed by Diwesh Chaurasia)

The following section was drafted for an earlier version of the project plan that included SFT and DPO stages. Those stages were not run in the final experiments; the text is included unchanged for transparency.

A.1 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT) serves as the initial stage of our post-training pipeline and provides the baseline behavior from which later reinforcement-based methods build. Before applying DPO or RFT, we fine-tune the base Llama 3.1–8B model on high-quality reasoning demonstrations to establish stable output formatting, mathematical coherence, and initial chain-of-thought capabilities.

Training Setup

For SFT, we fine-tuned the model using GSM8K-style reasoning samples formatted in our XML chain-of-thought structure:

```
<reasoning> ...  </reasoning>
<answer> ...  </answer>
```

This formatting is essential because later reinforcement learning stages rely on clean extraction of the reasoning and answer segments when computing rewards such as accuracy, formatting compliance, and chain-of-thought length. The model is fine-tuned using LoRA adapters (rank 64) applied to the quantized Llama 3.1–8B backbone, enabling training within the memory constraints of a single NVIDIA T4 GPU.

Training Objective

The SFT objective follows standard next-token prediction:

$$L_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim D_{\text{SFT}}} \sum_{t=1}^{T} \log p_\theta(y_t \mid x, y_{<t}).$$

This stage teaches the model to reproduce multi-step reasoning sequences, adhere to structured tagging conventions, and generate mathematically coherent solution steps. Importantly, SFT helps the model internalize the general style of chain-of-thought reasoning that later RL-based methods reinforce more explicitly.

Behavior After SFT

While SFT improves intermediate reasoning quality, we observe several limitations:

- The model often produces verbose or drifting reasoning traces.
- It treats the answer as open-ended text and may fail to terminate cleanly with the required `<answer>` tag.
- Final answers may still be missing or incorrect despite coherent reasoning steps.

These observations reflect known issues in SFT-only training, where imitation provides structure but cannot enforce correctness or stability.

Role of SFT in the Post-Training Pipeline

Although SFT does not induce "Aha Moment" reasoning, it plays a critical foundational role:

1. **Stable Initialization:** Reinforcement-based optimization such as GRPO is sensitive to initialization; SFT ensures the model already understands structured reasoning.
2. **Format Conditioning:** Reward functions rely on clean extraction of reasoning and answer segments; SFT teaches the XML structure used later by RFT.
3. **Preliminary Reasoning Ability:** SFT provides basic multi-step reasoning patterns that RL-based methods refine into more stable and concise forms.

Thus, SFT establishes the groundwork necessary for effective alignment and consistency in later stages such as DPO and RFT.

A.2 Direct Preference Optimization (DPO)

In this section, we describe how we applied **Direct Preference Optimization (DPO)** [24] as a post-training method for our small-scale language model (quantized Llama 3.1–8B with LoRA adapters), and our rationale for using it in the context of enhancing reasoning stability.

Motivation for DPO

Traditional RLHF pipelines require training a separate reward model and running reinforcement learning. DPO provides a simpler, more efficient approach by directly optimizing the model to prefer human-preferred outputs over less desirable ones using a pairwise preference loss. Given our compute limits (single T4 GPU, 4-bit quantization, LoRA rank 64), DPO is an appealing choice for improving reasoning behavior.

Implementation

We constructed a preference dataset consisting of GSM8K prompts paired with multiple candidate completions. These completions were ranked based on:

- correctness of the final answer,
- clarity and structure of the reasoning chain,
- compliance with the XML-style formatting.

Training used the standard DPO loss. For each prompt x, preferred output y_w, and non-preferred output y_l, we optimize:

$$\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big),$$

where π_{ref} is a frozen reference model.

We reused LoRA rank 64 applied to attention and feedforward layers to stay within memory constraints.

Observations and Role in "Aha Moment" Investigation

After DPO fine-tuning, we observed:

- clearer structure in chain-of-thought traces,
- more stable and consistent reasoning patterns,
- improved formatting adherence,
- better—but still limited—final-answer accuracy.

DPO encourages preferred reasoning styles but does not fundamentally expand reasoning capabilities. It improves discipline and stability, serving as a complementary step before reinforcement-based optimization such as RFT or GRPO.

Conclusion

DPO offered a lightweight, resource-efficient alignment method that strengthened clarity, structure, and consistency in outputs. While it does not directly induce "Aha Moment" reasoning or enable out-of-distribution generalization, it meaningfully stabilizes reasoning patterns in preparation for more advanced post-training stages.